

瞭解如何叫用已驗證的 Cloud Run 函式

來源：<https://codelabs.developers.google.com/codelabs/how-to-invoke-authenticated-cloud-function?hl=zh-tw>

步驟數：8

瞭解如何要求驗證才能叫用 Cloud Run 函式，以確保安全

目錄

1. [1. 簡介](#)
2. [2. 設定和需求](#)
3. [3. 建立及測試已驗證的 Cloud Run 函式](#)
4. [4. 情境 1：使用 gcloud ID 權杖](#)
5. [5. 情境 2：模擬服務帳戶](#)
6. [6. 情境 3：使用 Google 用戶端程式庫](#)
7. [7. 恭喜！](#)
8. [8. 清除所用資源](#)

步驟 1 · 約 0 分鐘

1. 簡介

總覽

[Cloud Run functions](#) 是一項簡易運算解決方案，可讓開發人員建立獨立的單一用途函式，透過 HTTPS 觸發或回應 [CloudEvents](#)，而不需管理伺服器或執行階段環境。如要進一步瞭解 Cloud Run 函式，請參閱[這篇網誌文章](#)。

控管 Cloud Run 函式調用的主要方法有兩種：[根據身分保護存取權](#)，以及使用[以網路為基礎的存取權控管](#)保護存取權。本程式碼研究室著重於第一種方法，並逐步說明 3 種情境，根據身分驗證確保函式叫用存取安全：

1. 使用 gcloud 身分識別權杖叫用函式，進行本機開發和測試
2. 在本機開發及測試時模擬服務帳戶，使用與正式版相同的憑證
3. 使用 Google 用戶端程式庫處理 Google Cloud API 的驗證，例如服務需要叫用函式時

課程內容

- 如何設定 Cloud Run 函式的驗證，以及確認驗證設定是否正確
 - 從本機開發環境叫用已驗證的函式，並提供 gcloud 身分識別的權杖
 - 如何建立服務帳戶並授予適當角色來叫用函式
 - 如何從本機開發環境模擬服務，並具備叫用函式的適當角色
-

步驟 2 · 約 0 分鐘

2. 設定和需求

必要條件

- 您已登入 Cloud Console
- 您先前已部署 HTTP 觸發的 Cloud Run 函式。 [查看快速入門範例](#)。
- (選用) 在第 3 種情境中，本程式碼研究室會以 [Node.js](#) 和 [npm](#) 為例，但您可以使用 [Google Auth 用戶端程式庫](#) 支援的任何執行階段。

啟用 Cloud Shell

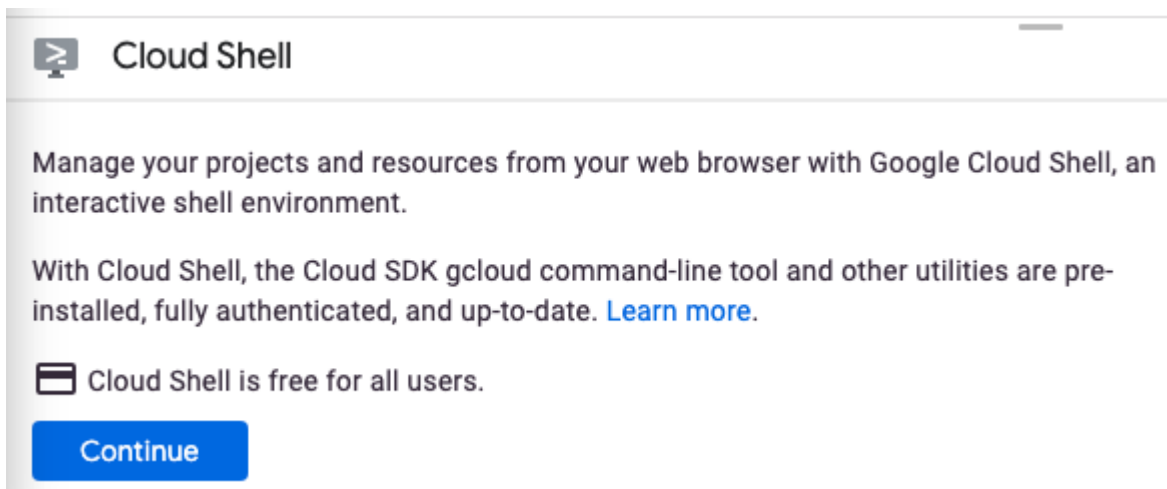
1. 在 Cloud 控制台，點選「啟用 Cloud Shell」圖示



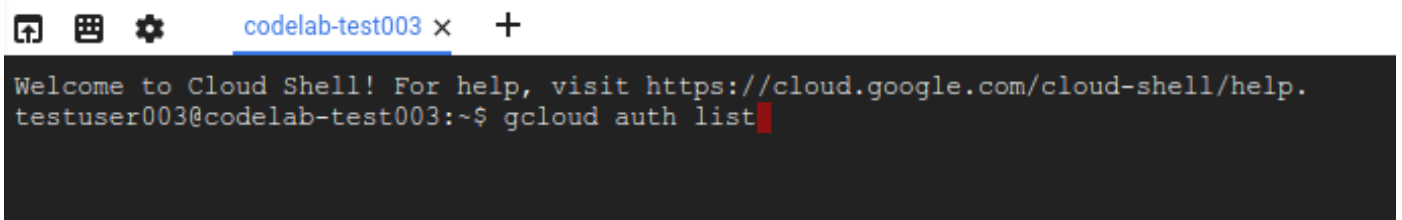
。



如果您是首次啟動 Cloud Shell，系統會顯示中繼畫面，說明這個指令列環境。如果出現中繼畫面，請按一下「繼續」。



佈建並連至 Cloud Shell 預計只需要幾分鐘。



這部虛擬機器已載入所有必要的開發工具，並提供永久的 5 GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。本程式碼研究室幾乎所有工作都可在瀏覽器上完成。

連至 Cloud Shell 後，您應該會看到驗證已完成，專案也已設為獲派的專案 ID。

2. 在 Cloud Shell 中執行下列指令，確認您已通過驗證：

```
gcloud auth list
```

指令輸出



```
Credentialed Accounts
ACTIVE  ACCOUNT
*       <my_account>@<my_domain.com>

To set the active account, run:
$ gcloud config set account `ACCOUNT`
```

注意： `gcloud` 指令列工具是 Google Cloud 中功能強大的整合式指令列工具，並已預先安裝於 Cloud Shell。你會發現它支援 Tab 鍵自動完成功能。詳情請參閱 [gcloud 指令列工具總覽](#)。

3. 在 Cloud Shell 中執行下列指令，確認 `gcloud` 指令知道您的專案：

```
gcloud config list project
```

指令輸出

```
[core]
project = <PROJECT_ID>
```

如未設定，請輸入下列指令手動設定專案：

```
gcloud config set project <PROJECT_ID>
```

指令輸出

```
Updated property [core/project].
```

注意： 如果您在 Cloud Shell 以外的環境完成本教學課程，請按照「[設定應用程式預設憑證](#)」一文的說明操作。

步驟 3 · 約 0 分鐘

3. 建立及測試已驗證的 Cloud Run 函式

如果要求驗證，叫用函式的主體就必須具備 Cloud Run 叫用者角色，否則函式會傳回 403 Forbidden 錯誤。本程式碼研究室會說明[如何將適當的 Invoker 角色授予主體](#)。

設定本機環境變數，簡化 gcloud 指令

首先，您將建立幾個環境變數，提升本程式碼研究室所用 `gcloud` 指令的可讀性。

```
REGION=us-central1
PROJECT_ID=$(gcloud config get-value project)
```

建立函式的原始碼

雖然本程式碼研究室使用 Node.js，但您可以使用 Google Auth 用戶端程式庫支援的任何執行階段。

首先，請建立目錄並 `cd` 到該目錄。

```
mkdir auth-function-codelab && cd $_
```

接著，建立 `package.json` 檔案。

```
touch package.json

echo '{
  "dependencies": {
    "@google-cloud/functions-framework": "^3.0.0"
  }
}' > package.json
```

接著，建立 `index.js` 來源檔案。

```
touch index.js

echo 'const functions = require("@google-cloud/functions-framework");

functions.http("helloWorld", (req, res) => {
  res.send(`Hello ${req.query.name || req.body.name || "World"}!`);
});' > index.js
```

建立已驗證的函式

以下是為 `nodejs20` 執行階段建立已驗證函式的步驟。不過，您可以使用 [Google Auth 用戶端程式庫](#) 支援的任何執行階段。

```
FUNCTION_NAME=authenticated-function-codelab  
ENTRY_POINT=helloWorld
```

如要將 Cloud Run 函式直接部署至 Cloud Run，請執行下列指令：

```
gcloud beta run deploy $FUNCTION_NAME \  
  --source . \  
  --function helloWorld \  
  --region $REGION \  
  --no-allow-unauthenticated
```

然後將函式網址儲存為環境變數，以供後續使用。

```
FUNCTION_URL="$(gcloud run services describe $FUNCTION_NAME --region $REGION --format  
'value(status.url)')"
```

如要部署為 Cloud Functions 第 2 代，請使用下列指令：

```
gcloud functions deploy nodejs-http-function \  
  --gen2 \  
  --runtime=nodejs20 \  
  --region=$REGION \  
  --source=. \  
  --entry-point=helloWorld \  
  --trigger-http \  
  --no-allow-unauthenticated
```

然後將函式網址儲存為環境變數，以供後續使用。

```
FUNCTION_URL="$(gcloud functions describe $FUNCTION_NAME --gen2 --region us-central1  
  --format='get(serviceConfig.uri)')"
```

嘗試以匿名來電身分叫用函式，確認函式是否需要驗證

您將在未經授權的情況下叫用函式，確認是否收到預期的 403 錯誤。

在指令列執行下列 `curl` 指令：

```
curl -i $FUNCTION_URL
```

您會看到下列結果：

```
<html><head>
<meta http-equiv="content-type" content="text/html; charset=utf-8">
<title>403 Forbidden</title>
</head>
<body text=#000000 bgcolor=#ffffff>
<h1>Error: Forbidden</h1>
<h2>Your client does not have permission to get URL <code>/</code> from this server.
</h2>
<h2></h2>
</body></html>
```

現在，您已準備好瞭解 3 種情境，並透過提供驗證來叫用函式。

步驟 4 · 約 0 分鐘

4. 情境 1：使用 gcloud ID 權杖

開發人員需要[在本機開發函式時進行測試](#)。在本節中，您將執行快速測試，確認函式已使用您的身分正確驗證。

執行下列指令，確認您已使用 `gcloud` 進行驗證：

```
gcloud auth list
```

您應該會在有效身分旁邊看到星號，例如：

```
Credentialed Accounts
ACTIVE  ACCOUNT

*       <my_account>@<my_domain.com>

To set the active account, run:
$ gcloud config set account `ACCOUNT`
```

如要進一步瞭解如何[設定 gcloud init](#) 和 [gcloud auth login](#)，請參閱相關說明文件。

接著，叫用函式並傳遞身分識別權杖。

```
curl $FUNCTION_URL -H "Authorization: bearer $(gcloud auth print-identity-token)"
```

現在您會看到結果：

```
Hello World!
```

疑難排解

如果收到 403 Forbidden 錯誤，請確認您的身分是否具備 [Cloud Run Invoker 角色](#)。您可以使用 [IAM 控制台](#)，驗證授予主體的角色。

雖然在開發期間使用自己的身分權杖，是測試函式的快速方法，但經過驗證的函式呼叫端必須具備適當的角色，否則呼叫端會收到 403 Forbidden 錯誤。

您應遵循[最小權限原則](#)，限制可叫用函式的身分和服務帳戶數量。在下一個情境中，您將瞭解如何建立新的服務帳戶，並授予適當的角色來叫用函式。

5. 情境 2：模擬服務帳戶

在這種情況下，您要**模擬** (即承擔權限) 服務帳戶，以便在本機開發及測試時叫用函式。模擬服務帳戶身分後，您就能使用與正式環境相同的憑證測試函式。

這樣做不僅能驗證角色，還能遵循最小權限原則，不必為了進行本機測試而將 Cloud Functions 叫用者角色授予其他身分。

在本程式碼研究室中，您將建立新的服務帳戶，該帳戶只具備叫用您在本程式碼研究室中建立的函式所需的角色。

建立新的服務帳戶

首先，您要建立幾個額外的環境變數，代表 gcloud 指令中使用的服務帳戶。

```
SERVICE_ACCOUNT_NAME="invoke-functions-codelab"
SERVICE_ACCOUNT_ADDRESS=$SERVICE_ACCOUNT_NAME@$PROJECT_ID.iam.gserviceaccount.com
```

接著，您將建立服務帳戶。

```
gcloud iam service-accounts create $SERVICE_ACCOUNT_NAME \
  --display-name="Cloud Run function Authentication codelab"
```

並授予服務帳戶 Cloud Run 叫用者角色：

```
gcloud run services add-iam-policy-binding $FUNCTION_NAME \
  --region=us-central1 \
  --member serviceAccount:$SERVICE_ACCOUNT_ADDRESS \
  --role='roles/run.invoker'
```

模擬服務帳戶來叫用函式

為此，您需要取得新建立服務帳戶的 ID 權杖，藉此模擬該帳戶。

新增模擬作業的必要角色

如要模擬服務帳戶，您的使用者帳戶必須具備**服務帳戶權杖建立者** ([roles/iam.serviceAccountTokenCreator](#)) 角色，才能為服務帳戶產生 ID 權杖。

您可以執行下列指令，將這個角色授予有效使用者帳戶：

```
ACCOUNT_EMAIL=$(gcloud auth list --filter=status:ACTIVE --format="value(account)")

gcloud iam service-accounts add-iam-policy-binding $SERVICE_ACCOUNT_ADDRESS \
  --member user:$ACCOUNT_EMAIL \
  --role='roles/iam.serviceAccountTokenCreator'
```

使用服務帳戶的 ID 權杖

請稍候幾分鐘，讓權限生效。現在您可以傳遞服務帳戶的 ID 權杖，叫用函式。

```
curl $FUNCTION_URL -H "Authorization: bearer $(gcloud auth print-identity-token --
impersonate-service-account $SERVICE_ACCOUNT_ADDRESS)"
```

您會看到以下內容：

```
WARNING: This command is using service account impersonation. All API calls will be
executed as [invoke-functions-codelab@<project-id>.iam.gserviceaccount.com].
```

```
Hello World!
```

6. 情境 3：使用 Google 用戶端程式庫

在本程式碼研究室的最後一部分，您將在本機執行小型服務，為服務帳戶產生 ID 權杖，然後使用 [Google Auth 用戶端程式庫](#) 和 [應用程式預設憑證 \(ADC\)](#)，以程式輔助方式呼叫函式。如要進一步瞭解 Google 用戶端程式庫，請參閱說明文件的 [用戶端程式庫說明一節](#)。

如果您想在本機 (例如在筆電、Cloud Shell 等) 編寫及測試函式，同時與其他 Google Cloud 資源 (例如 Cloud Storage、Vision API 等) 互動，就特別需要使用 ADC。在本範例中，您將瞭解如何讓服務叫用需要驗證的另一個函式。如要進一步瞭解 ADC 和本機開發作業，請參閱「[How to develop and test your Cloud Functions locally | Google Cloud Blog](#)」網誌文章。

執行 gcloud 指令，模擬服務帳戶

ADC 會根據應用程式環境自動尋找憑證，並使用這些憑證向 Google Cloud API 進行驗證。使用 `--impersonate-service-account` 旗標，即可模擬服務帳戶，並使用其身分向 Google Cloud API 進行驗證。

如要模擬服務帳戶，可以執行下列指令：

```
gcloud auth application-default login --impersonate-service-account=$SERVICE_ACCOUNT_ADDRESS
```

現在您是以該服務帳戶的身分執行 gcloud 指令，而非以您的身分。

建立及執行服務，叫用已驗證的函式

每個執行階段都有專屬的 [Google Auth 用戶端程式庫](#)，您可以安裝。本程式碼研究室會逐步引導您在本機建立及執行 Node.js 應用程式。

Node.js 的步驟如下：

1. 建立新目錄

```
mkdir local-dev && cd $_
```

2. 建立新的 Node.js 應用程式

```
npm init -y
```

3. 安裝 Google Auth 用戶端程式庫

```
npm install google-auth-library
```

4. 建立 `index.js` 檔案

5. 擷取 Cloud Run 函式的網址，並在下一個步驟中新增至程式碼。

```
echo $FUNCTION_URL
```

6. 在 `index.js` 中加入下列程式碼。請務必將 `targetAudience` 變數變更為 Cloud Run 函式網址。

index.js

```
// Cloud Functions uses your function's url as the `targetAudience` value

const targetAudience = '<YOUR-CLOUD-RUN-FUNCTION-URL>';

// For Cloud Functions, endpoint(`url`) and `targetAudience` should be equal

const url = targetAudience;

const { GoogleAuth } = require('google-auth-library');
const auth = new GoogleAuth();

async function request() {
  console.info(`request ${url} with target audience ${targetAudience}`);

  // this call retrieves the ID token for the impersonated service account
  const client = await auth.getIdTokenClient(targetAudience);

  const res = await client.request({ url });
  console.info(res.data);
}

request().catch(err => {
  console.error(err.message);
  process.exitCode = 1;
});
```

7. 執行應用程式

```
node index.js
```

您應該會看到「Hello World!」結果。

疑難排解

如果看到「Permission 'iam.serviceAccounts.getOpenIdToken' denied on resource (or it may not exist).」錯誤訊息，請稍候幾分鐘，等待「服務帳戶符記建立者」角色傳播。

如果您收到「Cannot fetch ID token in this environment」(無法在這個環境中擷取 ID 權杖) 錯誤，請使用 GCE，或將 `GOOGLE_APPLICATION_CREDENTIALS` 環境變數設為服務帳戶憑證 JSON 檔案，您可能忘記執行下列指令：

```
gcloud auth application-default login --impersonate-service-
account=$SERVICE_ACCOUNT_ADDRESS
```

步驟 7 · 約 0 分鐘

7. 恭喜！

恭喜您完成本程式碼研究室！

建議您參閱[這篇文章](#)，瞭解如何保護 Cloud Run 函式。

我們也建議您參閱這篇有關[使用 Cloud Run 函式進行本機開發作業](#)的網誌文章，瞭解如何在本地開發環境中開發及測試 Cloud Run 函式。

涵蓋內容

- 如何設定 Cloud Run 函式的驗證，以及確認驗證設定是否正確
 - 從本機開發環境叫用已驗證的函式，並提供 gcloud 身分識別的權杖
 - 如何建立服務帳戶並授予適當角色來叫用函式
 - 如何從本機開發環境模擬服務，並具備叫用函式的適當角色
-

步驟 8 · 約 0 分鐘

8. 清除所用資源

為避免產生非預期費用 (例如，這個 Cloud Function 遭非預期呼叫的次數，超過[免費層級每月 Cloud Run 函式呼叫配額](#))，您可以刪除 Cloud Function，或刪除在步驟 2 中建立的專案。

如要停止模擬服務帳戶，請使用您的身分重新登入：

```
gcloud auth application-default login
```

如要刪除 Cloud Run 函式，請前往 Cloud Run 函式 Cloud 控制台 (<https://console.cloud.google.com/functions/>)，並確認目前選取的專案是您在步驟 2 中建立的專案。

選取先前部署的 **my-authenticated-function**。然後按一下「刪除」。

如要刪除整個專案，請前往 <https://console.cloud.google.com/cloud-resource-manager>，選取您在步驟 2 中建立的專案，然後選擇「刪除」。刪除專案後，您必須在 Cloud SDK 中變更專案。如要查看所有可用專案的清單，請執行 `gcloud projects list`。
